UD3 · Persistencia con ORM

Acceso a Datos (0486) · DAM

De JDBC a JPA e Hibernate

Objetivo: persistir objetos Java en PostgreSQL sin escribir manualmente cada operación SQL.

Ruta de aprendizaje

1. Desfase objeto-relacional y mapeo
2. JPA frente a Hibernate
3. Instalación y configuración
4. Entidades y operaciones CRUD
5. Asociaciones, herencia y conversores
6. Ciclo de vida, caché y rendimiento
7. Actividad guiada y preguntas de repaso

Recordatorio: cómo trabajábamos con JDBC

- Connection: abrir comunicación con PostgreSQL.
- PreparedStatement: preparar SQL y parámetros.
- ResultSet: recorrer filas con next().
- DAO: transformar filas en objetos Cliente.
- Cada operación requiere SQL y mapeo explícitos.

```
String sql = "SELECT * FROM cliente";
try (Connection con = ...;
     PreparedStatement ps =
       con.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
  while (rs.next()) {
    // Crear Cliente manualmente
  }
}
```

¿Qué es ORM?

ORM = Object-Relational Mapping.

- Clase Java ↔ tabla SQL.
- Objeto ↔ fila.
- Atributo ↔ columna.
- Asociación entre objetos ↔ clave ajena o tabla intermedia.

El motor ORM genera y ejecuta SQL según las operaciones sobre objetos.

El desfase objeto-relacional

En Java pensamos en objetos, referencias, herencia y colecciones.

En SQL pensamos en tablas, filas, claves y relaciones.

El ORM traduce entre ambos mundos; no elimina sus diferencias.

```
Java                          PostgreSQL
class Cliente                  TABLE cliente
  id: Integer          ⇔       id INTEGER
  nombre: String       ⇔       nombre VARCHAR
  activo: boolean      ⇔       activo BOOLEAN
```

Comparativa JDBC frente a ORM

JDBC: SQL explícito, control directo, más código de mapeo.

ORM: entidades y contexto de persistencia, SQL generado, menos repetición.

Ambos utilizan JDBC por debajo en el ecosistema habitual de Java.

ORM no sustituye el conocimiento de SQL.

JPA no es Hibernate

- JPA / Jakarta Persistence: especificación estándar y API.
- Hibernate ORM: implementación que ejecuta el trabajo real.
- Programamos contra EntityManager (JPA).
- Hibernate se encarga del mapeo y las sentencias SQL.
- Alternativas de implementación: EclipseLink, entre otras.

Arquitectura del ejemplo

Interfaz Swing → Servicio / DAO JPA → EntityManager → Hibernate → Driver JDBC → PostgreSQL.

La interfaz no debería contener SQL ni gestionar sesiones de persistencia.

El proveedor traduce las operaciones sobre entidades a instrucciones SQL.

Instalación con Maven

Añadir dependencias de Hibernate ORM y el controlador JDBC PostgreSQL.

La API Jakarta Persistence puede llegar como dependencia transitiva; declararla explícitamente facilita identificar el estándar.

Comprobar versiones compatibles y resolver dependencias desde Maven.

```
<dependencies>
  <dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>VERSIÓN_COMPATIBLE</version>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>VERSIÓN_COMPATIBLE</version>
  </dependency>
</dependencies>
```

Unidad de persistencia

Archivo habitual: src/main/resources/META-INF/persistence.xml.

- Nombre de la unidad.
- Proveedor ORM.
- URL, usuario y contraseña.
- Configuración del esquema y trazas SQL.

Para clase, utilizar credenciales de desarrollo, no producción.

```
<persistence-unit name="tiendaPU"
 transaction-type="RESOURCE_LOCAL">
 <provider>
  org.hibernate.jpa.HibernatePersistenceProvider
 </provider>
 <properties>
  <property name="jakarta.persistence.jdbc.url"

value="jdbc:postgresql://localhost:5432/tienda_ra
miro"/>
 </properties>
</persistence-unit>
```

Propiedades esenciales de Hibernate

- hibernate.show_sql: observar las sentencias generadas.
- hibernate.format_sql: formatearlas para leerlas.
- hibernate.hbm2ddl.auto=validate: validar el esquema existente.
- create / create-drop: solo ejercicios controlados; pueden destruir datos.
- En producción: migraciones versionadas y gestión segura de secretos.

Primera entidad: Cliente

@Entity marca una clase persistente.

@Table establece la tabla.

@Id identifica la clave primaria.

@GeneratedValue indica cómo se genera el ID.

@Column configura columnas y restricciones.

```
@Entity
@Table(name = "cliente")
public class Cliente {
 @Id
 @GeneratedValue(strategy =
   GenerationType.IDENTITY)
 private Integer id;

 @Column(nullable = false)
 private String nombre;

 private String email;
 private boolean activo;

 protected Cliente() {}
 // getters y setters
}
```

¿Qué sucede con @Id y @GeneratedValue?

La base de datos identifica cada fila por su clave primaria.

IDENTITY: la columna del SGBD genera el valor al insertar.

SEQUENCE: Hibernate obtiene identificadores de una secuencia; puede permitir optimizaciones por lotes.

Elegir según PostgreSQL, la estrategia de inserción y el volumen.

JDBC INSERT vs EntityManager.persist

En JDBC escribíamos INSERT y recuperábamos generated keys.

Con JPA creamos la entidad y pedimos persistirla.

La ejecución efectiva del SQL depende del momento de flush y de la estrategia de ID.

```
EntityManager em = emf.createEntityManager();
EntityTransaction tx = em.getTransaction();
try {
 tx.begin();
 Cliente c = new Cliente();
 c.setNombre("Ana");
 c.setEmail("ana@ejemplo.com");
 c.setActivo(true);
 em.persist(c);
 tx.commit();
} catch (RuntimeException e) {
 if (tx.isActive()) tx.rollback();
 throw e;
} finally { em.close(); }
```

Leer por identificador: find

EntityManager.find(Cliente.class, id) busca una entidad por su clave primaria.

Devuelve la entidad o null si no existe.

Dentro del mismo contexto de persistencia, la misma identidad suele devolver la misma instancia.

```
Cliente cliente =
    em.find(Cliente.class, 5);

if (cliente != null) {
 System.out.println(cliente.getNombre());
} else {
 System.out.println("No encontrado");
}
```

Actualizar: detección de cambios

Una entidad gestionada puede modificarse mediante setters.

Hibernate detecta cambios (dirty checking) y sincroniza con SQL al hacer flush/commit.

No siempre hay que invocar explícitamente un UPDATE.

```
tx.begin();
Cliente c = em.find(Cliente.class, 5);
if (c != null) {
 c.setEmail("nuevo@ejemplo.com");
 c.setActivo(false);
}
tx.commit();
// Hibernate puede generar UPDATE
```

Eliminar: remove

Primero obtener una entidad gestionada.

Después llamar a remove dentro de una transacción.

La eliminación se sincroniza con la base de datos al hacer flush/commit.

```
tx.begin();
Cliente c = em.find(Cliente.class, 5);
if (c != null) {
 em.remove(c);
}
tx.commit();
```

Estados del ciclo de vida

TRANSIENT: objeto nuevo, no gestionado.

MANAGED: asociado al contexto de persistencia.

DETACHED: ya no está gestionado.

REMOVED: marcado para eliminación.

Entender estos estados ayuda a explicar persist, merge, find y remove.

JPQL: consultas sobre entidades

JPQL usa entidades y atributos Java, no nombres físicos de tablas y columnas.

La implementación traduce JPQL a SQL.

Para consultas complejas puede utilizarse SQL nativo.

```
List<Cliente> clientes = em.createQuery(
 "SELECT c FROM Cliente c " +
 "WHERE c.activo = :activo " +
 "ORDER BY c.nombre",
 Cliente.class)
 .setParameter("activo", true)
 .getResultList();
```

Asociaciones entre entidades

@ManyToOne: muchos pedidos pertenecen a un cliente.

@OneToMany: un cliente tiene varios pedidos.

@OneToOne: relación uno a uno.

@ManyToMany: relación muchos a muchos con tabla intermedia.

Elegir cuidadosamente propietario de la relación, claves y carga.

Ejemplo: Pedido → Cliente

La tabla pedido contiene cliente_id como clave ajena.

En Java trabajamos con la referencia a Cliente.

@JoinColumn indica la columna de unión.

```
@Entity
public class Pedido {
 @Id
 @GeneratedValue(strategy =
  GenerationType.IDENTITY)
 private Long id;

 @ManyToOne(fetch = FetchType.LAZY)
 @JoinColumn(name = "cliente_id")
 private Cliente cliente;
}
```

Carga LAZY y EAGER

LAZY: cargar una asociación cuando se necesita.

EAGER: intentar obtenerla de forma anticipada.

No confundir «cargar» con «hacer una única consulta»: el SQL real depende del proveedor y de la consulta.

Cuidado al acceder a asociaciones LAZY fuera del contexto abierto.

Problema N+1

Ejemplo: cargar 20 pedidos (1 SELECT) y consultar el cliente de cada pedido (hasta 20 SELECT adicionales).

Total posible: 21 consultas para una pantalla.

Soluciones: JOIN FETCH, EntityGraph o proyecciones según necesidad.

Inspeccionar SQL antes de optimizar.

```
SELECT p FROM Pedido p
JOIN FETCH p.cliente

// Una estrategia para recuperar
// pedidos con sus clientes
```

Herencia de entidades

SINGLE_TABLE: una tabla para toda la jerarquía, discriminador.

JOINED: tabla por clase con enlaces.

TABLE_PER_CLASS: tabla por clase concreta.

Compromiso entre normalización, número de joins y duplicación.

No elegir sin pensar en consultas y migraciones.

Tipos especiales: enum y convertidores

@Enumerated(EnumType.STRING) suele ser más estable que ORDINAL ante cambios de orden.

AttributeConverter permite mapear tipos de dominio a columnas.

Ejemplos: estado del pedido, código de moneda, valores especializados.

```
public enum Estado {
 PENDIENTE, ENVIADO, ENTREGADO
}

@Enumerated(EnumType.STRING)
@Column(name = "estado")
private Estado estado;
```

Objetos incrustados: @Embeddable

Agrupar calle, ciudad y código postal en una clase Direccion.

Con @Embedded sus campos viven en la tabla de la entidad propietaria.

Beneficio: un modelo Java más expresivo sin crear necesariamente otra tabla.

```
@Embeddable
public class Direccion {
 private String calle;
 private String ciudad;
 private String codigoPostal;
}

@Embedded
private Direccion direccion;
```

Caché de primer y segundo nivel

Primer nivel: contexto de persistencia; es parte del funcionamiento de JPA/Hibernate.

Segundo nivel: opcional, puede compartirse entre contextos y requiere configuración.

La caché puede reducir consultas, pero exige coherencia e invalidación.

No activarla indiscriminadamente: medir antes.

¿Qué herramienta elegir?

JPA/Hibernate: dominio con entidades y relaciones, CRUD habitual.

JDBC o mapeadores ligeros: SQL muy específico, informes, operaciones masivas o control fino.

Spring Data JPA añade abstracciones de repositorio por encima de JPA.

Pueden convivir distintas estrategias en una misma aplicación.